Pricing Baku's Homes from Scratch: Decision Trees and Pegasos SVMs on bina.az Listings

Emil Mammadov Gasham Huseynli Orkhan Nuriyev

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We study sale listings from bina.az, Azerbaijan's largest property portal, and ask two questions: how much is a listing worth (Task A, regression on $\log(1+\text{price})$), and is it a premium or a standard listing (Task B, split at the training median)? We implement a decision tree and a linear soft-margin SVM trained with Pegasos entirely in NumPy, and compare them with scikit-learn under a leakage-safe protocol: flat-grouped splits, out-of-fold target encodings, validation-only tuning, bootstrap uncertainty and nested cross-validation. Our tree matches scikit-learn's to three decimals (Task A R^2 0.856; Task B ROC-AUC 0.972). Our Pegasos SVM reaches F1 0.908, about 1.6 points below an exact RBF `SVC` (0.924) but roughly 300× faster to train. A closed-form ridge baseline wins Task A (R^2 0.870), which tells us that most of the price signal is close to linear once the features are engineered. The hardest listings are luxury houses above 800k AZN, which every model under-predicts. Code: `github.com/Emilsdeyta/ml-final-project-bina-az` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, regression, classification, target leakage, real-estate price prediction.

I. Introduction

Buying a flat in Baku means reading hundreds of listings and guessing what is a fair price. A model that estimates the price from a handful of facts (area, rooms, floor, district, coordinates) is useful both to buyers and to the portal itself. It is also a good test bench for two classic algorithms, because the data is real, messy and full of traps.

This project had two goals. First, to understand a decision tree and a support-vector machine from the inside by writing both ourselves in NumPy, with scikit-learn allowed only as a baseline. Second, to run a comparison we can defend: no target leakage, no tuning on the test set, and uncertainty on every number. Our contributions are:

- a cleaned, de-duplicated, leakage-audited modelling set built from the raw bina.az dump, with every decision documented;
- a from-scratch CART tree (Gini, entropy and MSE) and a from-scratch Pegasos SVM with a random-Fourier-feature extension;
- a uniform evaluation harness (metrics, bootstrap, nested CV, error analysis) that scores our models and scikit-learn's with the same code;
- a one-command pipeline that reproduces every number and figure in this paper.

II. Related Work and Background

Decision trees. We follow the CART formulation [2]: at each node we pick the feature and threshold that maximise the impurity decrease, using Gini or entropy for classification and variance (MSE) for regression, and stop with depth and leaf-size rules.

SVMs and Pegasos. A soft-margin SVM [3] minimises the regularised hinge loss. Pegasos [1] optimises this primal directly with stochastic sub-gradient steps and a $1/(\lambda t)$ step size, so no quadratic-programming solver is needed. To go beyond a linear boundary we use random Fourier features [4], which approximate an RBF kernel and let us keep the same linear solver.

Baselines and practice. We compare with scikit-learn [5]. Our ridge baseline uses the closed-form normal equations [6]. Our leakage rules follow the usual guidance on target leakage [7], and our uncertainty estimates use the bootstrap [8].

III. Data and Preprocessing

A. Cleaning

The Kaggle dump has 100,775 scraped rows, but the same listing appears on many scrape days. We keep only the latest scrape of each listing (63,773 rows), drop listings whose price moved by more than 2× between scrapes (re-listings and typos), keep residential sales (57,796), and apply fixed sanity bounds of 15–1500 m^2 and 200–15,000 AZN/m^2. Duplicate content with different identifiers is removed too, which leaves **57,480** listings. Numeric columns that arrive as strings (area, floor written as "5 / 12", land area in *sot*) are parsed with regular expressions, and coordinates outside Azerbaijan are set to missing. All 20 cleaning decisions, with evidence, are in `DECISIONS.md`.

B. Target leakage

Two columns restate the target: `total_price` (correlation with `price` of 1.0000) and `unit_price`, whose product with area also correlates 1.0000 with price. A model trained on either would look spectacular and be useless in practice, so both are dropped and `clean()` asserts that they are gone. We also remove identifiers and free text (owner and shop names, address, description, image URLs) and post-listing popularity signals such as views and VIP flags, which are not known when a price is set. A column that is 99.8% empty (building type) is dropped as well.

C. Features and splits

Missingness here is structural: houses have no storeys and flats have no land plot. We therefore impute structurally, and

Table 1. Dataset summary after cleaning.

Property	Value
Raw rows	100,775
Rows after cleaning	57,480
Features used	27
Train / Val / Test split	40,236 / 8,622 / 8,622
Premium threshold (train median)	208,000 AZN
Premium rate (train / val / test)	0.500 / 0.500 / 0.500

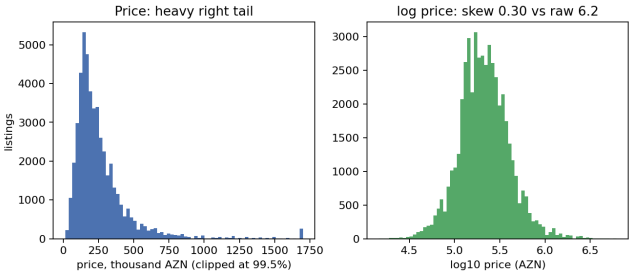


Figure 1. Prices are heavy-tailed; log price is close to symmetric.

use group-wise medians (rooms by category and area quintile, coordinates by district) instead of one global median. The final **27 features** cover size, floors, location, mortgage and deed flags, repair status and building category. For location we use the district's log-frequency and a smoothed target encoding, plus the mean, standard deviation and distance of log price per m^2 among each listing's 20 nearest *training* listings.

Every statistic (imputation values, clip bounds, scaler, tier threshold, encodings) is learned on training rows only. The two target-based features are computed out-of-fold for training rows, so a row never sees its own label. Because the same flat is often posted several times, we group re-posted copies of one physical flat (same location, area, rooms, floors, land and category) and keep each group in a single split. The result is 56,391 flats and zero shared across splits.

Tasks. Task A predicts $\log(1+\text{price})$ and we invert with $\exp(\cdot)-1$; the log transform cuts the skewness from 6.3 to 0.3 (Fig. 1). Task B labels a listing *premium* if its price is above the training median, so the classes are balanced by construction.

IV. Method

A. Decision tree (from scratch)

Our tree is a binary CART tree in NumPy with a scikit-learn-style `fit`/`predict` API. For a node with samples S split into S_L and S_R, the split score is the impurity decrease

$$\Delta I = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R), \tag{1}$$

with I equal to Gini $1-\sum_k p_k^2$, entropy $-\sum_k p_k \log_2 p_k$, or the variance $\frac{1}{N}\sum_i (y_i-\bar{y})^2$ for regression. For every feature we sort the values, scan the midpoints between consecutive distinct values, and keep the best *(feature, threshold)* pair; class counts and running sums make each scan linear after the sort. Growth stops on `max_depth`, `min_samples_split`, `min_samples_leaf` or `min_impurity_decrease`, and a stopping rule that fires at the root really does produce a single leaf (we test this). Leaves predict the majority class or the mean, and `predict_proba` returns leaf class frequencies. Feature importance is the total impurity decrease credited to each feature.

B. Support-vector machine (from scratch)

Our SVM minimises the regularised hinge-loss primal for labels $y_i \in \{-1,+1\}$:

$$J(\mathbf{w},b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n} \max\big(0,\, 1-y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big). \tag{2}$$

At step t we draw one example i, set $\eta_t = 1/(\lambda t)$ and update

$$\mathbf{w} \leftarrow \begin{cases} (1-\eta_t\lambda)\,\mathbf{w} + \eta_t y_i \mathbf{x}_i & y_i(\mathbf{w}^\top \mathbf{x}_i + b) < 1, \\ (1-\eta_t\lambda)\,\mathbf{w} & \text{otherwise}, \end{cases} \tag{3}$$

with the bias updated by $\eta_t y_i$ on margin violations. After each step we project $\mathbf{w}$ back onto the ball of radius $1/\sqrt{\lambda}$, the stability trick from the Pegasos paper. Features are standardised with training statistics first, because the hinge loss is scale-sensitive. The final model uses 20,000 steps; we log J every 200 steps to check convergence. Afterwards we report $\|\mathbf{w}\|$, the margin width $2/\|\mathbf{w}\|$, and the support vectors, i.e. training points with $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \le 1$.

For a non-linear boundary we chose **random Fourier features** over kernelised Pegasos. They map $\mathbf{x}$ to a cosine embedding with $\phi(\mathbf{x})^\top \phi(\mathbf{x}') \approx e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|^2}$ and keep the cheap linear update, which stays fast on 40k rows. We use 200 random components and sweep the bandwidth γ.

C. Baselines

Two "know-nothing" floors (mean predictor, majority class) tell us what a model has to beat. A closed-form ridge regression, $\hat{\mathbf{w}} = (X_c^\top X_c + \alpha I)^{-1} X_c^\top y_c$ with an unpenalised intercept, serves as a Task A baseline.

V. Experimental Setup

Metrics. Task A: RMSE, MAE and R^2 on log price, plus R^2, MAE and median absolute percentage error (MdAPE) in AZN. Task B: accuracy, precision, recall, F1, ROC-AUC and PR-AUC. All metrics are implemented in NumPy in one module that never imports scikit-learn, and its unit tests check every metric against scikit-learn.

Tuning. All hyperparameters are chosen on the validation split only. The tree grids vary depth (6, 8, 10, 12), minimum leaf size (15, 30) and, for Task B, the criterion (Gini or entropy); because the search runs on a random 8,000-row subsample of the training data for speed, the final trees are refit on the full training set. For the SVM we sweep 18 values of λ on a log grid from 10^{-5} to 1 and 12 values of γ from 10^{-3} to 10^2. The tier threshold, imputation values and scaler come from the training rows. The test set is touched once, to report results.

Uncertainty. We report a bootstrap standard deviation (300 resamples of the test set) and, separately, 5-fold *nested* cross-validation on train+val, where all preprocessing and encodings are re-fitted inside each fold and the folds respect the flat groups. A plain CV on pre-encoded features would leak labels across folds.

Baselines and matching. The scikit-learn trees use the same criterion, depth and leaf size as ours. `SGDClassifier` (hinge loss, $\alpha = \lambda$) is the matched *linear* SVM; `SVC` uses an RBF kernel with $C = 1/\lambda$ and is therefore a more flexible model, not a like-for-like one. Every model, ours or not, is refit and scored by the same harness. All randomness uses seed 42, and Python 3.11, NumPy 1.26.4 and scikit-learn 1.4.2 are pinned.

VI. Results

A. Task A: price regression

Table 2 shows the test results. Both trees and ridge are far ahead of the mean predictor (RMSE 0.63), and our tree is indistinguishable from scikit-learn's: R^2 0.856 for both, and the MAE differs by 160 AZN on a scale of 49,000. The surprise is that ridge regression is *better* than both trees (RMSE 0.226 vs. 0.239; the gap is several times the bootstrap standard deviation of 0.003–0.004), and nested CV agrees (ridge R^2 0.874, tree 0.863). After target encoding and geo-neighbour features, price is nearly linear in the inputs, and a tree has to approximate that with axis-aligned steps.

B. Task B: price tier

Table 3 summarises Task B. Our tree again matches scikit-learn (F1 0.914 and AUC 0.972 for both). Our Pegasos SVM reaches F1 0.908 and AUC 0.968, within one to two standard deviations of the matched linear baseline `SGDClassifier` (F1 0.913), which is what we expect from two solvers for the same objective. The exact RBF `SVC` is best (F1 0.924, AUC 0.981), which we read as the value of a non-linear boundary rather than a better optimiser, but it takes 306 s to train against about one second for Pegasos. Nested CV gives the same ranking (tree F1 0.913, Pegasos 0.910, SVC 0.925).

C. Tuning and the bias–variance story

The validation numbers show the trade-off clearly (Table 4). For regression, validation R^2 climbs from 0.812 at depth 6 to 0.840 at depth 10, then slips to 0.838 at depth 12: the tree stops gaining from extra splits and starts to fit noise. For classification at leaf size 15, AUC falls steadily with depth (0.958 at depth 6, 0.955 at depth 12), so deeper trees only overfit, and a larger leaf (30) is the best Gini setting (0.960). Gini and entropy are essentially tied (0.9565 vs. 0.9574 at depth 10, leaf 15). Validation and test scores of our selected models also stay close: the final regression tree scores validation R^2 0.860 and test R^2 0.856, and the classification tree scores AUC 0.969 on validation and 0.972 on test, and the nested CV means agree. We saw no sign of a large generalisation gap.

SVM. Figure 2 shows the λ sweep. Validation F1 is best at $\lambda = 3.4 \times 10^{-2}$ (0.905 in the sweep, 0.910 for the final 20,000-step model). Figure 3 shows the objective J during training, and our unit tests check that it decreases on separable data. The learned weight norm is $\|\mathbf{w}\| = 1.67$, giving a margin width of 1.20, and 13,618 of the 40,236 training points (34%) lie on or inside the margin. That a third of the data are support vectors is plausible for a median split, since a flat at 205k and one at 211k get different labels but look alike. The random-Fourier-feature variant did *not* help: its

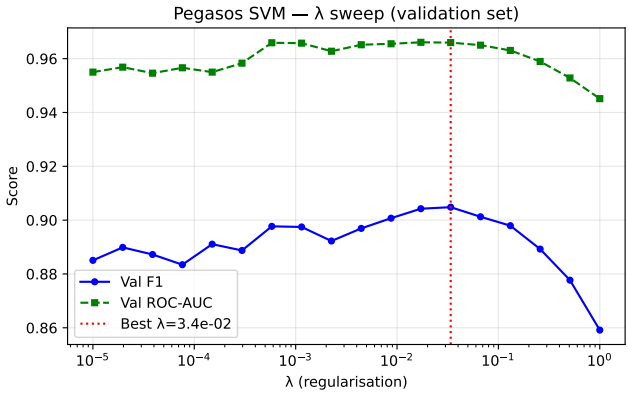


Figure 2. Pegasos λ sweep: validation F1 and ROC-AUC. The dotted line marks the selected λ.

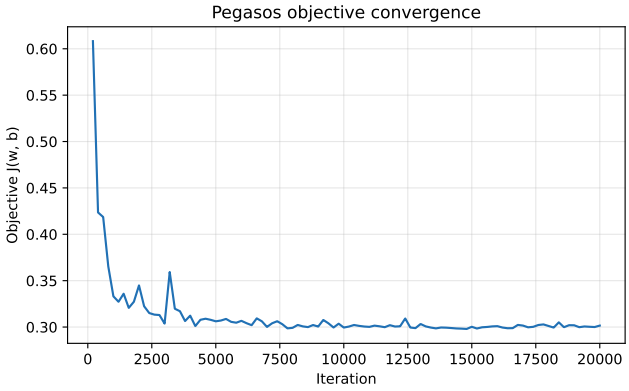


Figure 3. Pegasos training objective $J(\mathbf{w}, b)$, logged every 200 steps for the final model.

best validation F1 was 0.849 at $\gamma = 8.1 \times 10^{-3}$, below the linear model's 0.910, so we kept the linear one as our main SVM. With only 200 components, a single global bandwidth and standardised mixed features, the approximation is probably too coarse; we did not tune it further.

D. Feature importance and weights

Impurity-decrease importance is dominated by area: `area_m2` carries 0.62 of the tree's importance for regression and 0.65 for classification. The next two are location signals, namely the neighbour-price feature (0.21 for regression) and the district target encoding (0.12 for regression, 0.19 for classification). Everything else is small. The SVM tells a consistent story from its weights: its largest weights are on the log-area, neighbour-price and district features.

VII. Error Analysis

The analysis below uses the best Task A model by test RMSE (our ridge) and, for Task B, the best classifier by test F1 (scikit-learn's `SVC`).

Task A. Errors are not uniform. Mid-priced listings (100k–400k AZN) have a median absolute percentage error of about 10%, but the model regresses toward the mean (Fig. 4). It overestimates cheap listings (bias +15% below 100k AZN) and underestimates luxury ones (−27% above 800k AZN, MdAPE 32%). Houses are the hardest category (MdAPE 22.7% against about 10% for flats), and the

Table 2. Task A (price regression), test set. ±: bootstrap std over 300 resamples. CV: 5-fold nested cross-validation on train+val. Best per column in bold.

Model	RMSE (log)	MAE (log)	R^2 (log)	MAE (AZN)	CV R^2	Fit (s)
Mean predictor	0.629 ± 0.006	0.488 ± 0.004	−0.000	127,850	−0.000	0.00
Ridge, normal eq. (ours)	**0.226 ± 0.003**	**0.157 ± 0.002**	**0.870 ± 0.004**	**48,280**	**0.874**	0.01
Decision tree (ours)	0.239 ± 0.004	0.163 ± 0.002	0.856 ± 0.004	49,516	0.863	12.5
scikit-learn `DecisionTreeRegressor`	0.238 ± 0.004	0.162 ± 0.002	0.856 ± 0.004	49,353	0.863	0.9

Table 3. Task B (price tier), test set. ±: bootstrap std. CV: 5-fold nested cross-validation. Best per column in bold.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV F1	Fit (s)
Majority class	0.500	0.000	0.000	0.000	0.500	0.000	0.00
Decision tree (ours)	0.914 ± 0.003	0.916 ± 0.004	0.912 ± 0.004	0.914 ± 0.003	0.972 ± 0.002	0.913	65.4
scikit-learn `DecisionTreeClassifier`	0.914 ± 0.003	0.916 ± 0.004	0.912 ± 0.004	0.914 ± 0.003	0.972 ± 0.002	0.913	0.7
Pegasos SVM (ours)	0.906 ± 0.003	0.889 ± 0.005	0.928 ± 0.004	0.908 ± 0.003	0.968 ± 0.002	0.910	1.1
scikit-learn `SVC` (RBF)	**0.924 ± 0.003**	**0.920 ± 0.004**	**0.929 ± 0.004**	**0.924 ± 0.003**	**0.981 ± 0.001**	**0.925**	306.2
scikit-learn `SGDClassifier` (hinge)	0.912 ± 0.003	0.902 ± 0.004	0.924 ± 0.004	0.913 ± 0.003	0.969 ± 0.002	0.912	0.1

Table 4. Tree tuning on the validation split (8,000-row training subsample). Task A: R^2; Task B: ROC-AUC.

Task	Criterion	depth / leaf	Val. score
A	MSE	6 / 15	0.8119
A	MSE	8 / 15	0.8346
A	MSE	**10 / 15**	**0.8400**
A	MSE	12 / 15	0.8383
A	MSE	10 / 30	0.8360
B	Gini	6 / 15	0.9576
B	Gini	8 / 15	0.9573
B	Gini	10 / 15	0.9565
B	Gini	12 / 15	0.9550
B	Gini	**10 / 30**	**0.9600**
B	Entropy	10 / 15	0.9574

largest absolute errors are dominated by under-predicted large houses (see `worst_errors_taskA.csv`); Binagadi has the highest district-level error. Price there depends on land, view and construction quality, none of which we capture well (we did not use the free-text description), and a log-scale model compresses the upper tail.

Task B. Errors sit near the tier threshold, as one would expect for a median split. The error rate is 11.4% in the 200k–400k bucket and 8.1% in 100k–200k, but below 1% for listings above 400k and about 0.4% below 100k. Most mistakes are therefore listings that are genuinely close to the threshold.

VIII. Discussion and Limitations

What worked. The biggest wins came from data work rather than model choice. De-duplication removes the same listing from several splits, and grouping by physical flat keeps re-posts apart. Out-of-fold encodings let us use location without leaking labels. Our tree and Pegasos SVM do what the reference implementations do, which is the best evidence that they are correct. Our tree matches scikit-learn's despite differences in tie-breaking and floating-point thresholds (we use midpoints between sorted values); the main cost

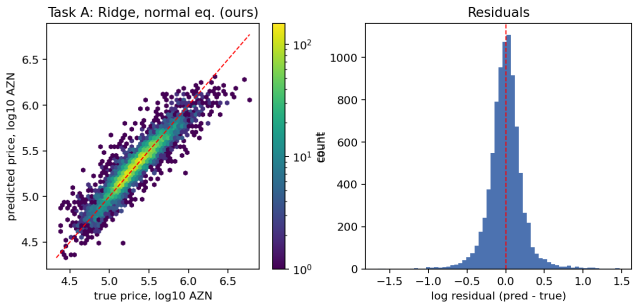


Figure 4. Task A: predicted vs. true price and log residuals for the best model.

is speed (65 s against 0.7 s) because scikit-learn's splitter is compiled.

What did not. A tree does not beat ridge on Task A, and our non-linear SVM extension did not beat the linear one. Both are honest negative results, and we report them as they are.

Limitations. (i) Prices are asking prices, not transaction prices. (ii) We have 27 tabular features and no text or photo information. (iii) Luxury houses are rare and systematically under-predicted; two-stage or quantile models would be the next thing to try. (iv) "Premium" means above the training median, so Task B is a relative notion. (v) Tree tuning used an 8,000-row subsample and a small grid, and the SVM's RFF extension used only 200 components, so both could be improved with more search. (vi) `SVC` is a non-linear model and not a like-for-like comparison with our linear SVM; `SGDClassifier` is the fair one.

IX. Conclusion

We built a decision tree and a Pegasos SVM from scratch, ran them through a leakage-safe protocol on 57k bina.az listings, and matched scikit-learn on the tree and came within about half an F1 point of the matched linear SVM. Ridge regression wins Task A, and an exact RBF SVM wins Task B, so our two from-scratch models are best seen as a faithful, fast and interpretable pair, not as record holders. With more

time we would add land-area interactions and text features, try quantile or two-stage models for the luxury segment, and build a random forest on top of our own trees. Everything in this paper is reproduced by `python -m src.run_all -strict`.

Tools and Acknowledgements

The tree, SVM and evaluation code are our own NumPy implementations; scikit-learn is used only as a baseline and as a test oracle. We checked every number in the text against `results.json` produced by our pipeline, and we are responsible for all content. The dataset is the Kaggle *binaaz-sale-project* by sehriyarmemmedli, built on listings from bina.az.

References

[1] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[2] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[3] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[4] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems 20*, 2007.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[7] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[8] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.